

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 17
Balanced Trees

Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline

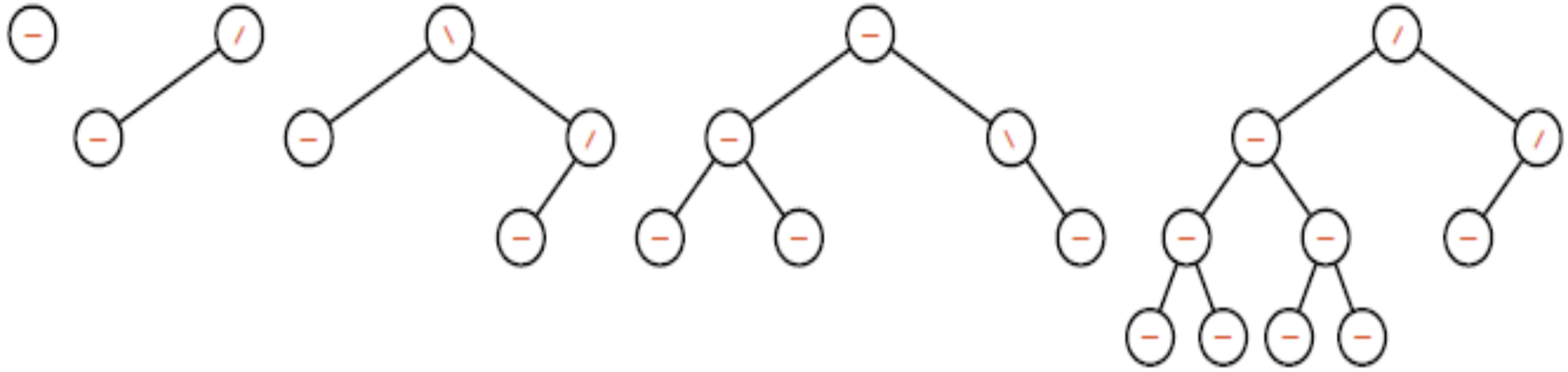
- AVL Trees
- Red-Black Trees*

Balanced Binary Search Trees: AVL Trees

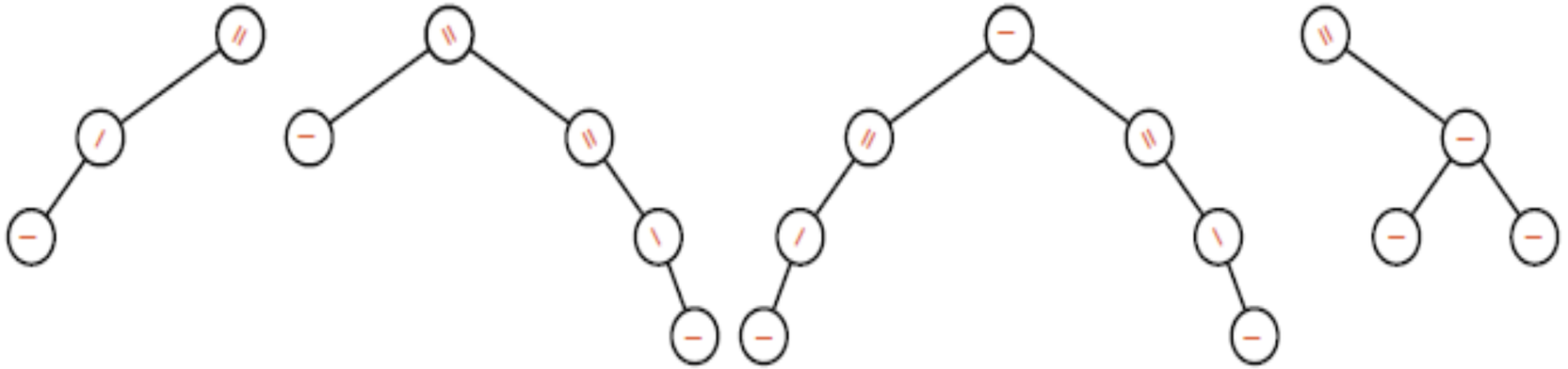
平衡二叉树

- An *AVL* tree is a binary search tree in which the heights of the left and right subtrees of the root differ by at most 1 and in which the left and right subtrees are again AVL trees.
- With each node of an AVL tree is associated a *balance factor*（平衡因子，左子树的高度减去右子树的高度） that is *left higher*, *equal*, or *right higher* according, respectively, as the left subtree has height greater than, equal to, or less than that of the right subtree.
- The name AVL comes from the discoverers of this method, G. M. Adel'son-Vel'ski and E. M. Landis. The method dates from 1962.

AVL Tree Examples



AVL trees

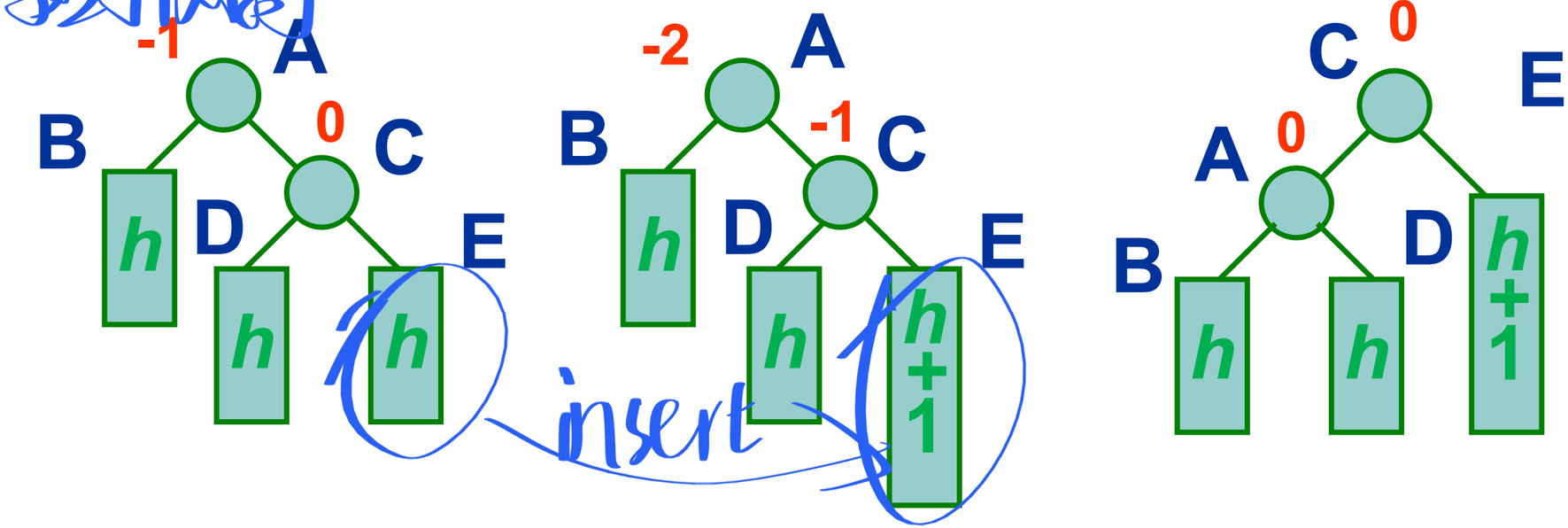


non-AVL trees

Insertions into an AVL Tree

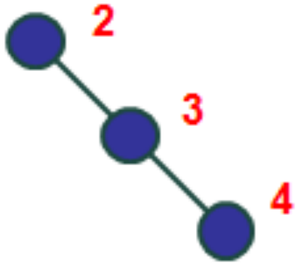
- 如果在一棵AVL树中插入一个新结点，就有可能造成失衡，此时必须重新调整树的结构，使之恢复平衡。我们称调整平衡过程为平衡旋转(rotation)。
- 对于失去平衡的二叉树可以归纳为4种类型：
  - RR型：单向左旋平衡处理
  - LL型：单向右旋平衡处理
  - RL型：双向旋转，先右后左
  - LR型：双向旋转，先左后右

RR Type: Left Rotation



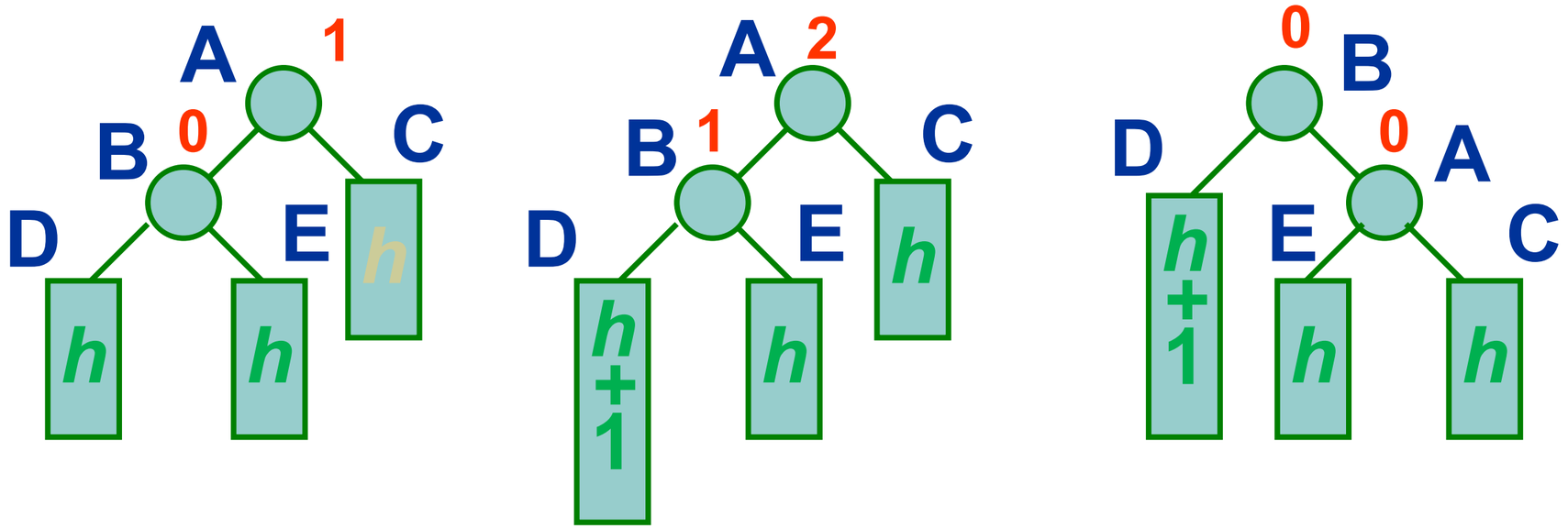
(a) 插入到C的右子树　　(b)右子树的右子树偏高　　(c)左旋转

- 结点A是插入后失去平衡的最高层结点，
  插入后A的右子树的右子树偏高，
  则进行左旋转。



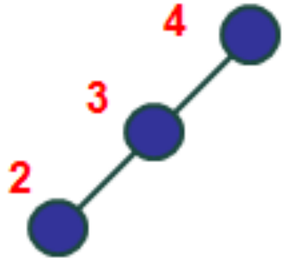
RR旋转

LL Type: Right Rotation



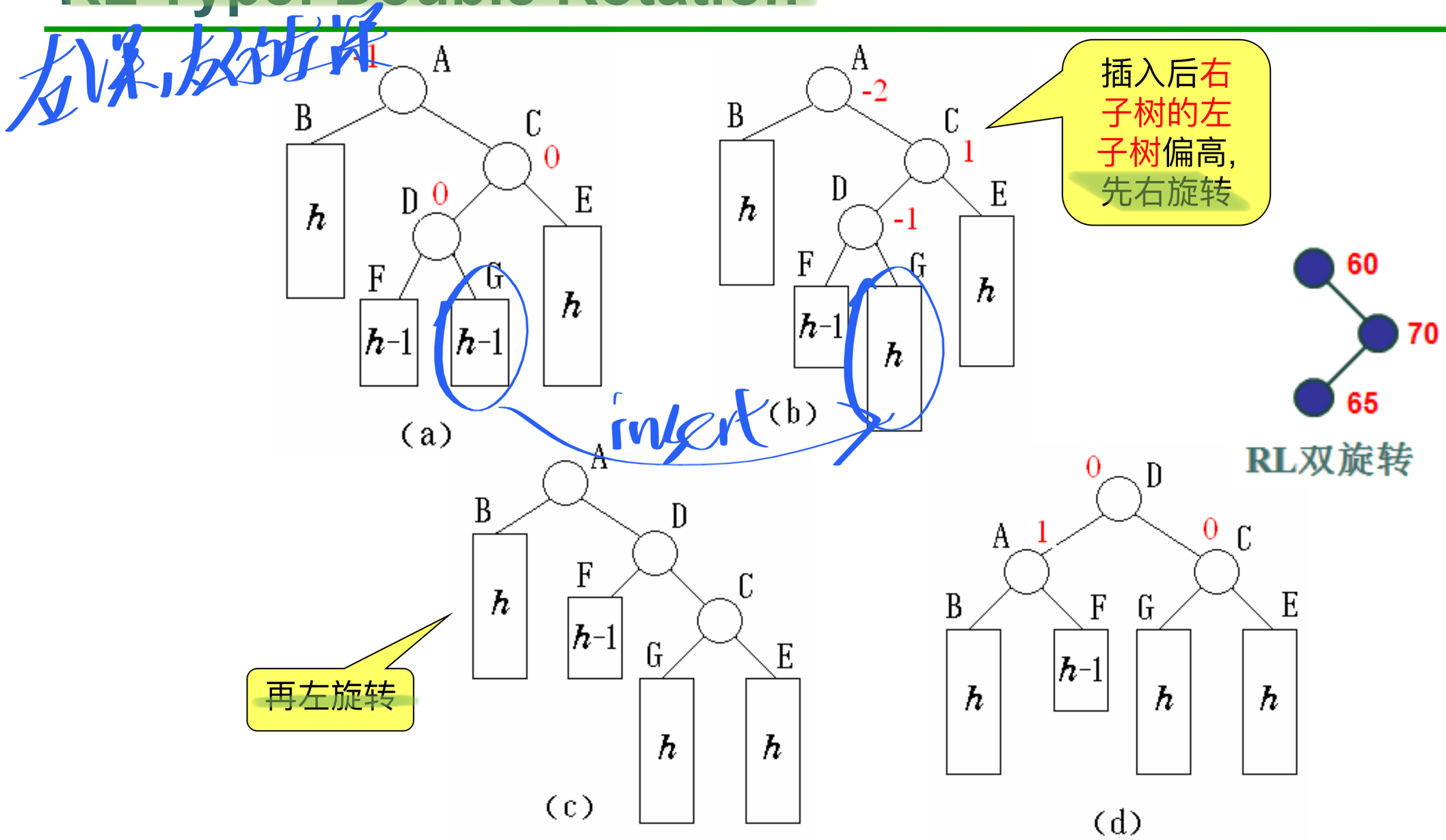
(a) 插入到B的左子树　　(b)左子树的左子树偏高　　(c)右旋转

- 结点A是插入后失去平衡的最高层结点，
  插入后A的左子树的左子树偏高，
  则进行右旋转。

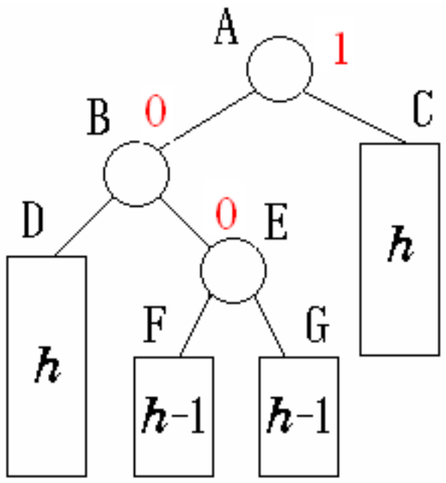


LL旋转

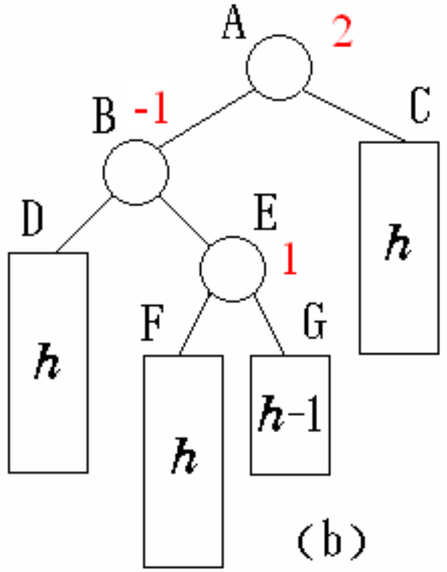
RL Type: Double Rotation



LR Type: Double Rotation

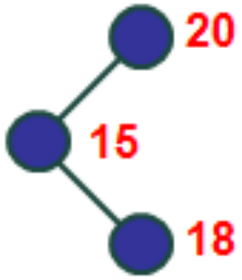


(a)

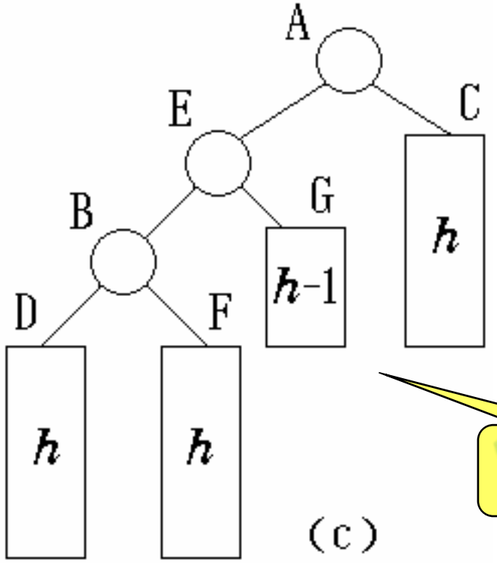


(b)

插入后左子树的右子树偏高，先左旋转

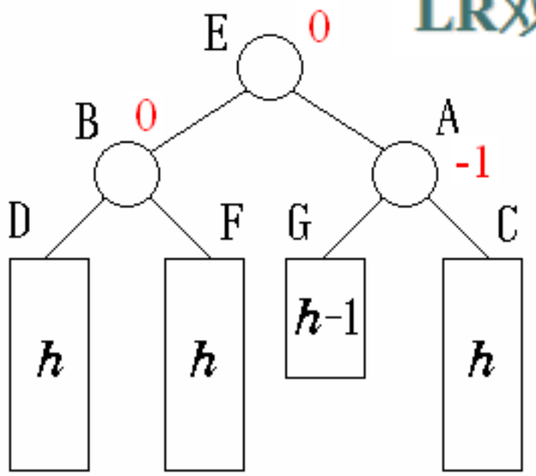


LR双旋转



(c)

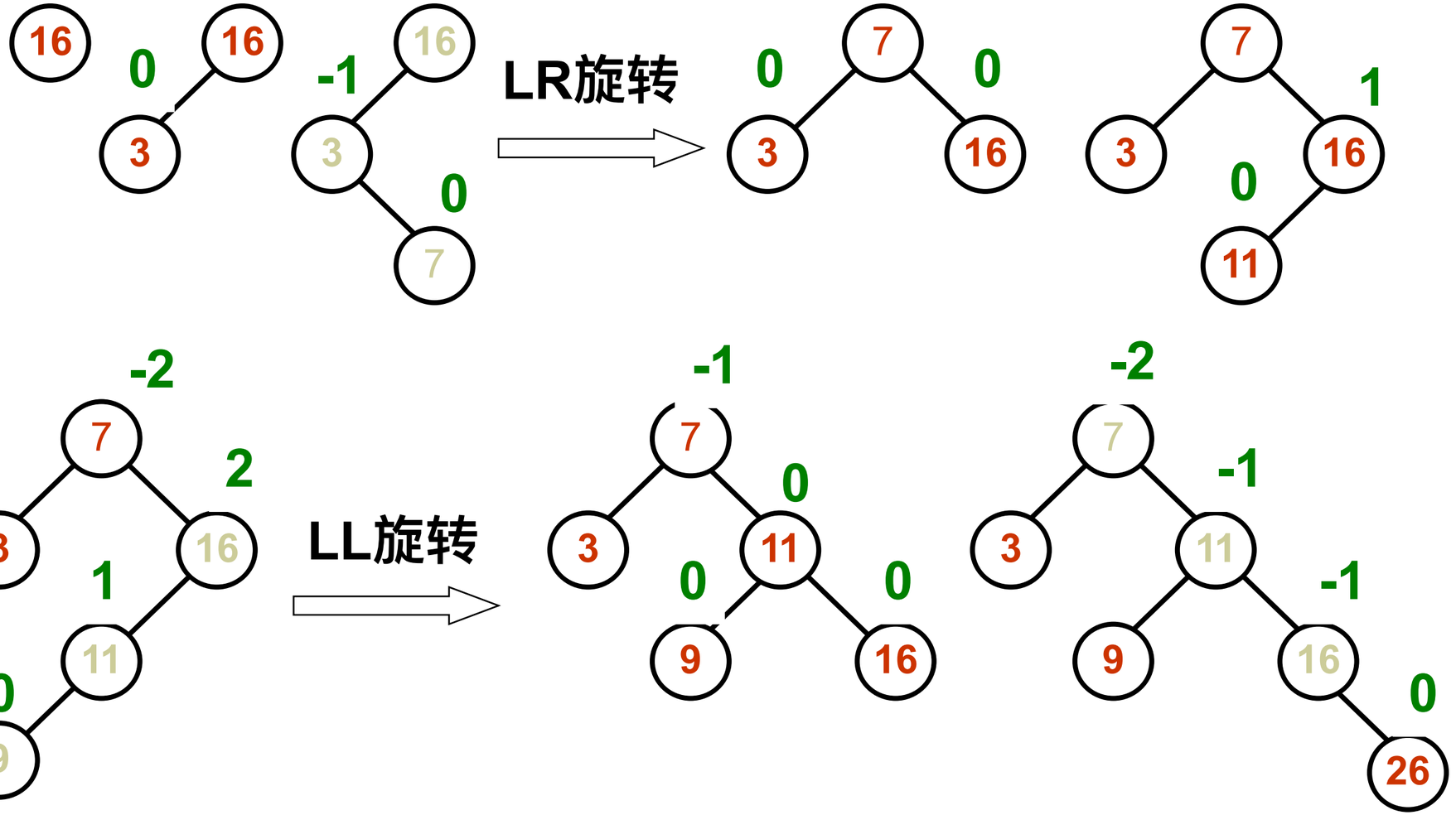
再右旋转



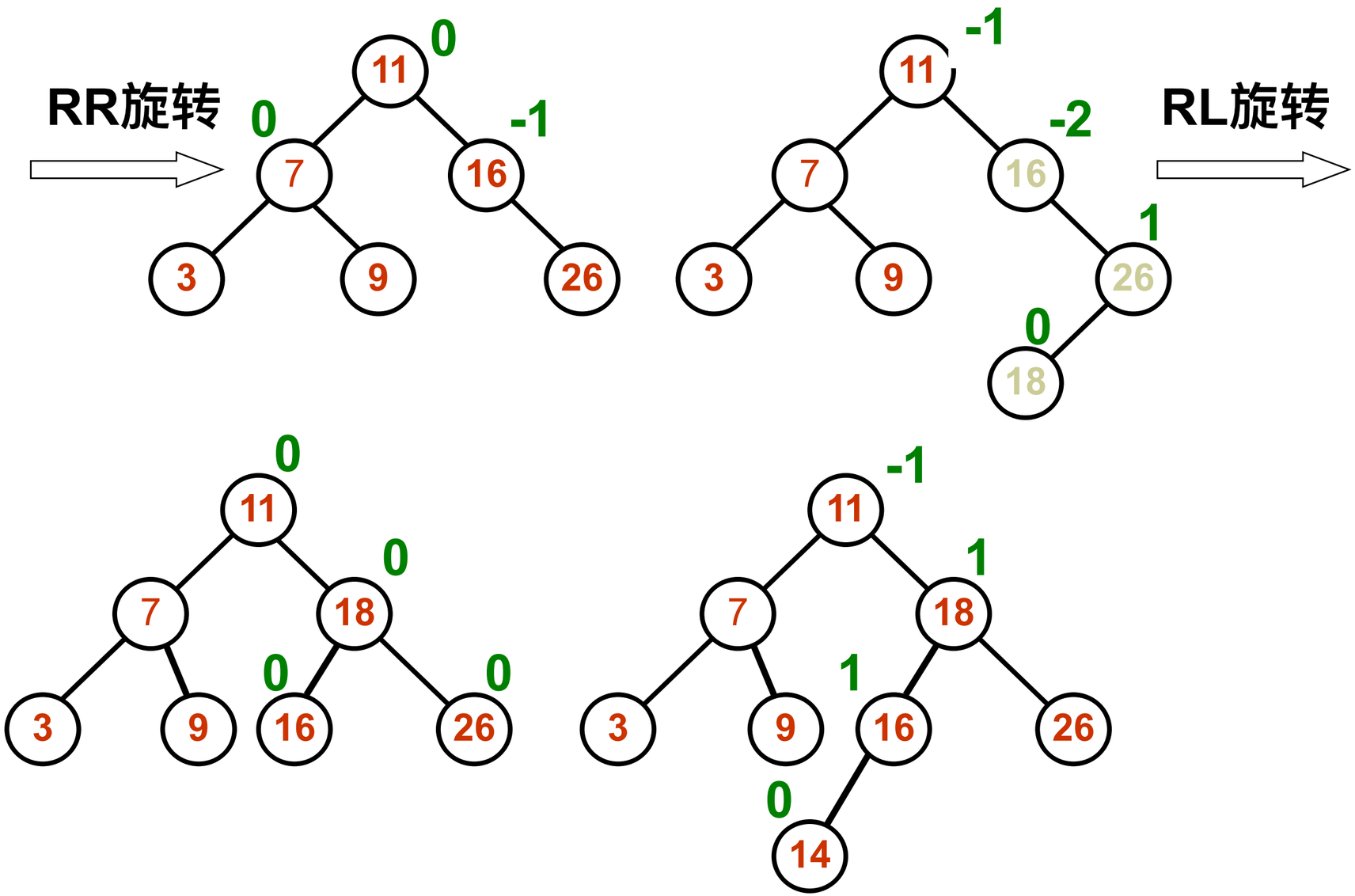
(d)

An AVL Example

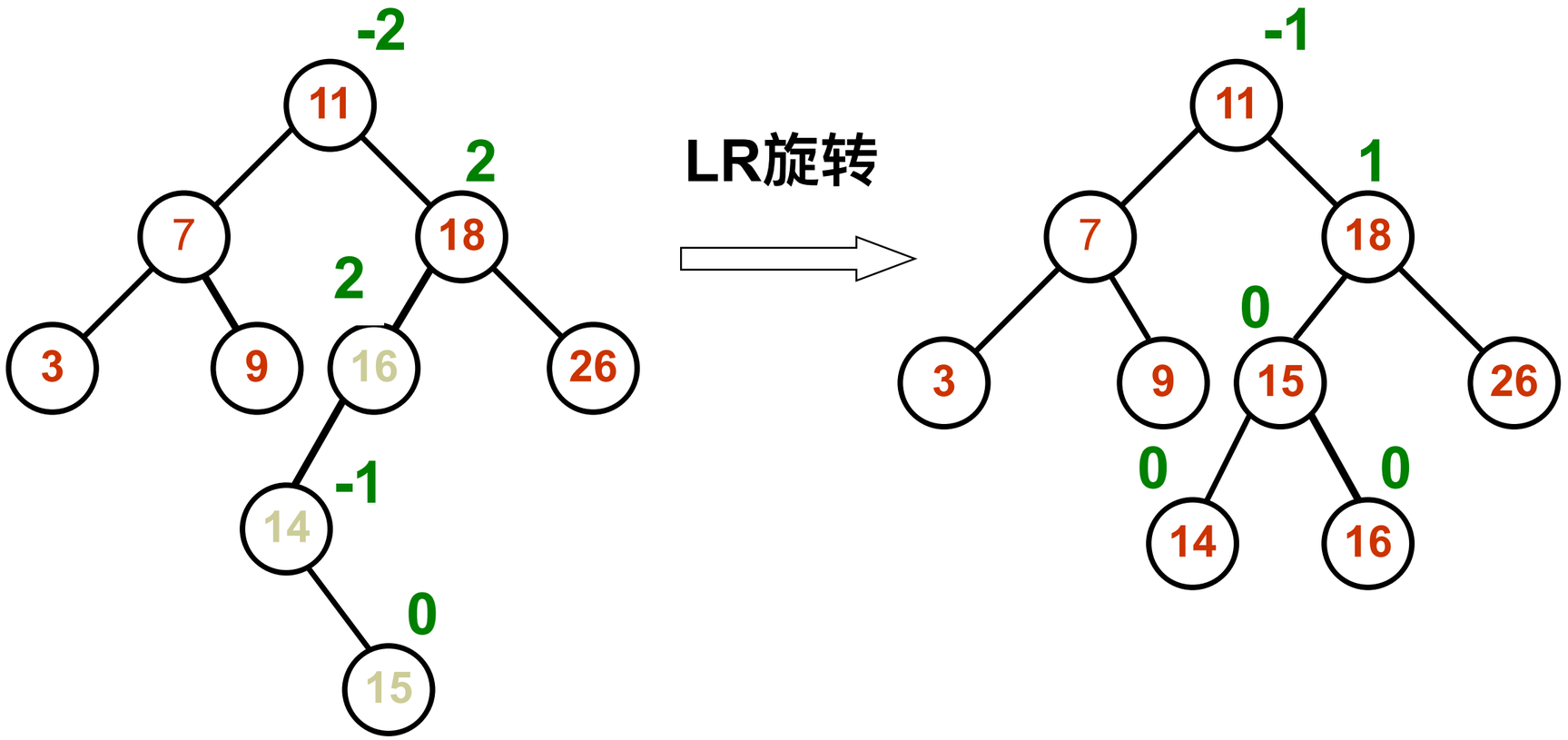
- 输入关键码序列为 { 16, 3, 7, 11, 9, 26, 18, 14, 15 }，插入和调整过程如下。



An AVL Example



An AVL Example



Exercise

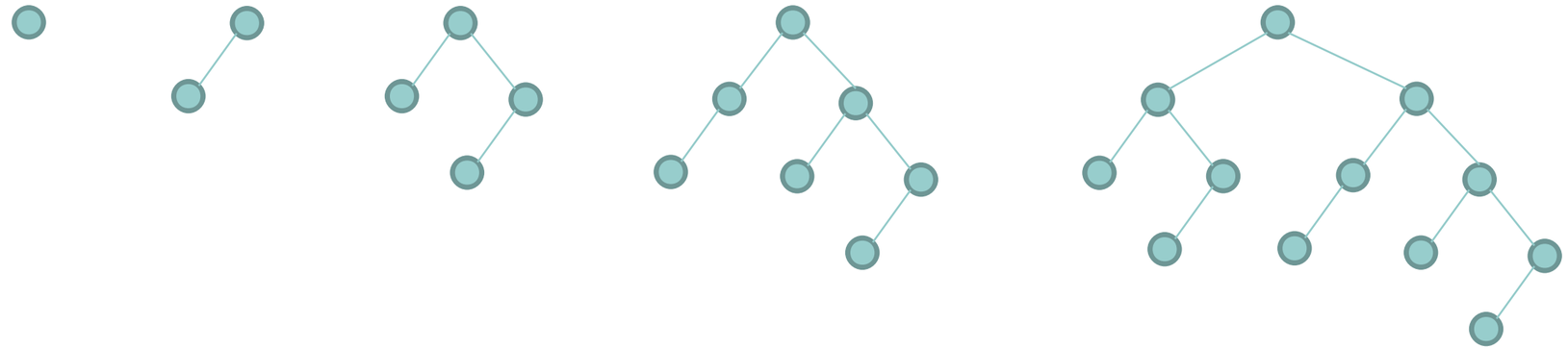
- In each of the following, insert the keys, in the order shown, to build them into an AVL tree.
  - (1) A, Z, B, Y, C, X.
  - (2) A, Z, B, Y, C, X, D, W, E, V, F.
  - (3) M, T, E, A, Z, G, P.
  - (4) A, B, C, D, E, F, G, H, I, J, K, L.
  - (5) A, V, L, T, R, E, I, S, O, K.

Analysis of AVL Trees

- The number of recursive calls to insert a new node can be as large as the height of the tree.
- At most one (single or double) rotation will be done per insertion.
- A rotation improves the balance of the tree, so later insertions are less likely to require rotations.
- It is very difficult to find the height of the *average* AVL tree, but the worst case is much easier. The worst-case behavior of AVL trees is essentially no worse than the behavior of random search trees.

Worst-Case AVL Trees

- To find the maximum height of an AVL tree with n nodes, we instead find the minimum number of nodes that an AVL tree of height h can have.
- Let F_h be such a tree, with left and right subtrees F_l and F_r. Then one of F_l and F_r, say F_l, has height $h - 1$ and the minimum number of nodes in such a tree, and F_r has height $h - 2$ with the minimum number of nodes.



F_0 F_1 F_2 F_3 F_4

Worst-Case AVL Trees

- If we write $|T|$ for the number of nodes in a tree T, we then have the recurrence relation for Fibonacci trees:

$$|F_h| = |F_{h-1}| + |F_{h-2}| + 1$$

  where $|F_0| = 1$ *and* $|F_1| = 2$.

- By the evaluation of Fibonacci numbers

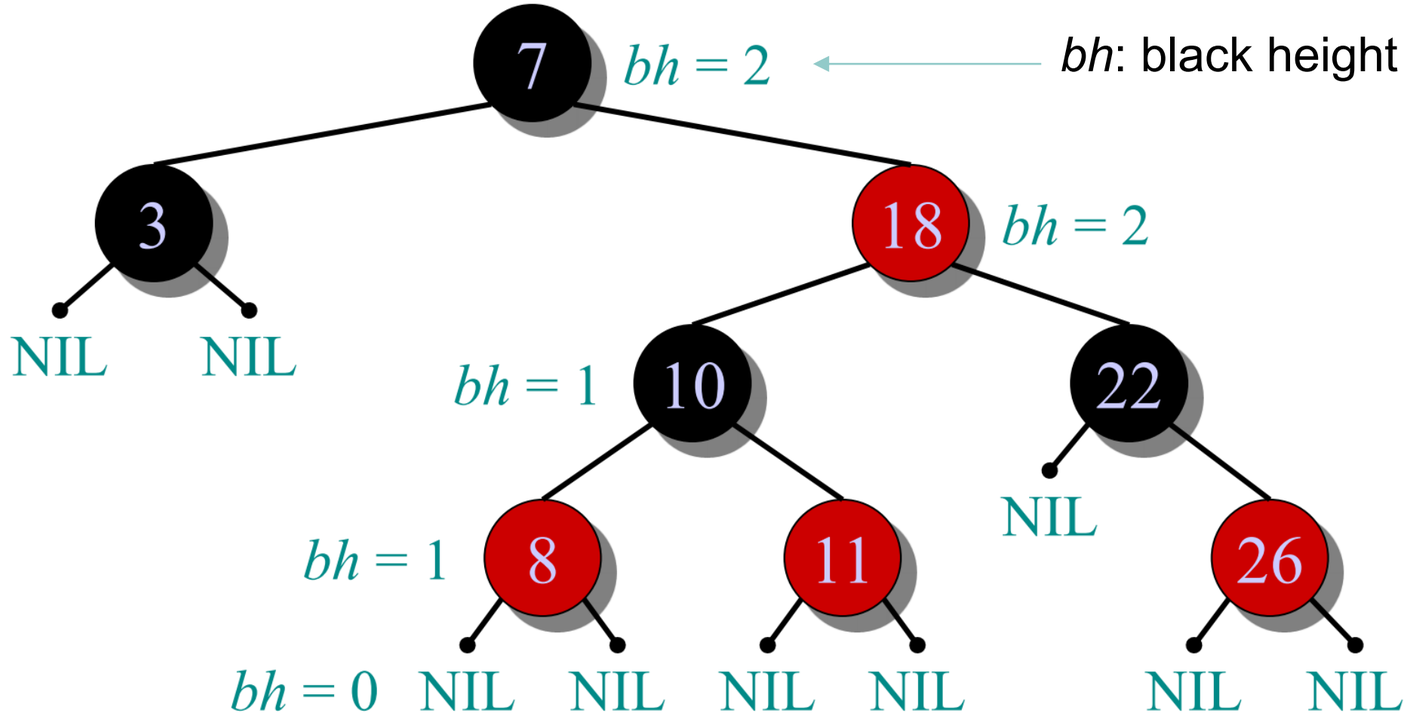
$$h \approx 1.44 \lg |F_h|$$

- This means that the sparsest (最稀疏的) possible AVL tree with n nodes has height about $1.44 \lg n$.

Red-Black Trees

- A red-black tree is a kind of self-balancing binary search tree where each node has an extra bit, and that bit is often interpreted as the color (red or black).
- These colors are used to ensure that the tree remains balanced during insertions and deletions. Although the balance of the tree is not perfect, it is good enough to reduce the searching time and maintain it around O(log *n*) time, where *n* is the total number of elements in the tree.
- This tree was invented in 1972 by Rudolf Bayer.
- STL “set” and “map” are usually implemented as Red-black trees.

Example of a Red-black Tree



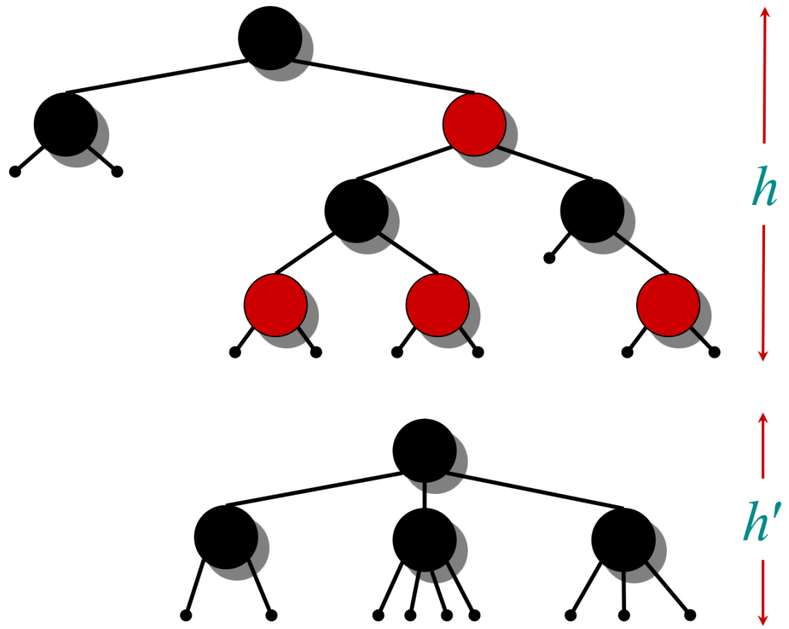
1. Every node is either red or **black**.
2. The root and leaves (NIL) are **black**.
3. If a node is red, then its parent is **black**.
4. All simple paths from any node x to a descendant leaf have the same number of **black** nodes.

Height of a Red-black Tree

- Theorem:

  A red-black tree with n keys has height $h <= 2 \lg(n + 1)$

- Intuition:
  - Merge red nodes into their black parents.
  - We have $h' >= h/2$, since at most half the leaves on any path are red.



Modifying Operations

- The operations INSERT and DELETE cause modifications to the red-black tree:
  - The operation itself
  - Color changes
  - Restructuring the links of the tree: “Rotations”.

Analysis

- The queries SEARCH, MIN, MAX, SUCCESSOR, and PREDECESSOR all run in O(lg n) time on a red-black tree with n nodes.
- INSERT requires O(lg n) with O(1) rotations.

Red-Black Tree Explanation

- https://www.bilibili.com/video/BV1854y187eo

Thank you!

